

```
In [1]: import torch
import torchvision
from torch.utils import data
from torchvision import transforms
import matplotlib.pyplot as plt
```

```
In [2]: def get_dataloader_workers():
        """使用 4 个子进程并行读取数据。"""
        return 4
```

```
In [3]: def load_data_fashion_mnist(batch_size, resize=None): # resize=None 表示是否调整图片大小
        """下载Fashion-MNIST数据集，然后将其加载到内存中"""
        trans = [transforms.ToTensor()]
        if resize:
            trans.insert(0, transforms.Resize(resize)) # 在转换为 tensor 前先调整大小
        trans = transforms.Compose(trans) # 组合所有的操作
        mnist_train = torchvision.datasets.FashionMNIST(
            root="./data", train=True, transform=trans, download=True)
        mnist_test = torchvision.datasets.FashionMNIST(
            root="./data", train=False, transform=trans, download=True)
        return (data.DataLoader(mnist_train, batch_size, shuffle=True,
                                num_workers=get_dataloader_workers()),
                data.DataLoader(mnist_test, batch_size, shuffle=False,
                                num_workers=get_dataloader_workers()))
```

```
In [4]: batch_size = 256
        train_iter, test_iter = load_data_fashion_mnist(batch_size)
```

```
100.0%
100.0%
100.0%
100.0%
```

## 1. 初始化模型参数

展平每个图像（28×28），将它们视为长度为784的向量。因为我们的数据集有10个类别，所以网络输出维度为10。

```
In [5]: num_inputs = 784
        num_outputs = 10

        W = torch.normal(0, 0.01, size=(num_inputs, num_outputs), requires_grad=True)
        b = torch.zeros(num_outputs, requires_grad=True)
```

## 2. 定义 softmax 操作

1. 对每个项求幂（使用exp）；
2. 对每一行求和（小批量中每个样本是一行），得到每个样本的规范化常数；
3. 将每一行除以其规范化常数，确保结果的和为1。

```
In [6]: def softmax(X):
        X_exp = torch.exp(X)
        partition = X_exp.sum(1, keepdim=True)
        return X_exp / partition # 这里应用了广播机制
```

## 3. 定义模型

```
In [7]: def net(X):
        return softmax(torch.matmul(X.reshape((-1, W.shape[0])), W) + b)
```

## 4. 定义损失函数

```
In [8]: def cross_entropy(y_hat, y):
        return - torch.log(y_hat[range(len(y_hat)), y])
```

## 5. 分类精度

```
In [9]: def accuracy(y_hat, y):
        """计算预测正确的数量"""
```

```

if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
    # len(y_hat.shape) 判断维度是否大于 1; y_hat.shape[1] 判断类别是否大于 1
    y_hat = y_hat.argmax(axis=1) # 取每一行的最大值所在的位置
    cmp = y_hat.type(y.dtype) == y
    return float(cmp.type(y.dtype).sum())

```

```

In [10]: class Accumulator:
def __init__(self, n): # 构造函数: n表示需要几个变量
    self.data = [0.0] * n

def add(self, *args): # 定义累加函数
    self.data = [a + float(b) for a, b in zip(self.data, args)]

def reset(self):
    self.data = [0.0] * len(self.data)

def __getitem__(self, idx): # 原本只支持 metric.data[0] 来访问, 现支持 metric[0]
    return self.data[idx]

```

| 模块        | 训练模式    | 评估模式       |
|-----------|---------|------------|
| Dropout   | 随机丢弃神经元 | 关闭 Dropout |
| BatchNorm | 更新均值方差  | 使用保存的均值方差  |

```

In [11]: def evaluate_accuracy(net, data_iter):
    """计算在指定数据集上模型的精度"""
    if isinstance(net, torch.nn.Module): # 判断 net 是否是 PyTorch 的神经网络模型。
        net.eval() # 将模型设置为评估模式
    metric = Accumulator(2)
    """
    metric[0] → 正确预测数量
    metric[1] → 总预测数量
    """
    with torch.no_grad():
        for X, y in data_iter:
            metric.add(accuracy(net(X), y), y.numel())
    return metric[0] / metric[1]

```

## 6. 训练

```

In [12]: def train_epoch(net, train_iter, loss, updater):
    """训练模型一个迭代周期, 并返回平均损失和训练精度。"""
    if isinstance(net, torch.nn.Module):
        net.train()
    metric = Accumulator(3) # 损失之和、正确预测数、样本数
    for X, y in train_iter:
        y_hat = net(X)
        l = loss(y_hat, y)
        if isinstance(updater, torch.optim.Optimizer): # 判断更新器是不是 PyTorch 官方优化器
            updater.zero_grad()
            l.mean().backward()
            updater.step()
        else:
            l.sum().backward()
            updater(X.shape[0])
        metric.add(l.sum().detach(), accuracy(y_hat, y), y.numel())
    return metric[0] / metric[2], metric[1] / metric[2]

```

```

In [13]: def sgd(params, lr, batch_size):
    with torch.no_grad():
        for param in params:
            param -= lr * param.grad / batch_size
            param.grad.zero_()

```

```

In [14]: lr = 0.1
num_epochs = 10
updater = lambda batch_size: sgd([W, b], lr, batch_size)

for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(net, train_iter, cross_entropy, updater)
    test_acc = evaluate_accuracy(net, test_iter)
    print(f'Epoch {epoch + 1}/{num_epochs}: '
          f'train loss {train_loss:.4f}, train acc {train_acc:.4f}, '
          f'test acc {test_acc:.4f}')

```

Epoch 1/10: train loss 0.7887, train acc 0.7461, test acc 0.7910  
 Epoch 2/10: train loss 0.5686, train acc 0.8135, test acc 0.8069  
 Epoch 3/10: train loss 0.5259, train acc 0.8248, test acc 0.8087  
 Epoch 4/10: train loss 0.5013, train acc 0.8319, test acc 0.8196  
 Epoch 5/10: train loss 0.4843, train acc 0.8371, test acc 0.8267  
 Epoch 6/10: train loss 0.4726, train acc 0.8406, test acc 0.8265  
 Epoch 7/10: train loss 0.4645, train acc 0.8437, test acc 0.8257  
 Epoch 8/10: train loss 0.4578, train acc 0.8456, test acc 0.8305  
 Epoch 9/10: train loss 0.4524, train acc 0.8469, test acc 0.8176  
 Epoch 10/10: train loss 0.4468, train acc 0.8483, test acc 0.8296

## 7. 预测

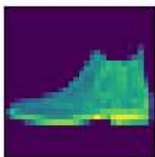
```
In [15]: def get_fashion_mnist_labels(labels):
  """返回Fashion-MNIST数据集的文本标签"""
  text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
    'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
  return [text_labels[int(i)] for i in labels]

In [16]: def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5): #@save
  """绘制图像列表"""
  figsize = (num_cols * scale, num_rows * scale) # 计算画布的大小
  _, axes = plt.subplots(num_rows, num_cols, figsize=figsize) # 创建多个图片窗口, axes 表示所有子图对象
  axes = axes.flatten() # 将 axes 展平成一维数组
  for i, (ax, img) in enumerate(zip(axes, imgs)): # zip() 实现配对得到 (ax, img); enumerate() 给列表增加索引
    if torch.is_tensor(img):
      # 图片张量
      ax.imshow(img.numpy()) # matplotlib 只认识 numpy, 不认识 tensor
    else:
      # PIL 图片
      ax.imshow(img)
    # 隐藏坐标轴
    ax.axes.get_xaxis().set_visible(False)
    ax.axes.get_yaxis().set_visible(False)
    # 显示标题
    if titles:
      ax.set_title(titles[i])
  return axes

In [17]: def predict_ch3(net, test_iter, n=6):
  """预测标签"""
  for X, y in test_iter: # 取一个batch
    break
  trues = get_fashion_mnist_labels(y) # 真实标签
  preds = get_fashion_mnist_labels(net(X).argmax(axis=1)) # 预测标签
  titles = [true + '\n' + pred for true, pred in zip(trues, preds)] # 标题
  show_images(X[:n].reshape(n, 28, 28), 1, n, titles=titles[:n]) # 显示图片

predict_ch3(net, test_iter)
```

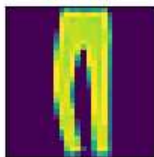
ankle boot  
ankle boot



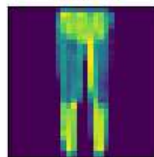
pullover  
pullover



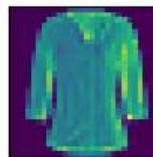
trouser  
trouser



trouser  
trouser



shirt  
shirt



trouser  
trouser

